

CS 121 Data Structures in C

02 Algorithm Analysis

Outline

- Introduction of algorithm and data structure
- Algorithm analysis
 - ♦ Space complexity
 - ♦ Time complexity
- Asymptotic notation
 - ♦ Big-Oh
 - ♦ Big-Omega
 - ♦ Big-Theta

Introduction

- What is an Algorithm?
 - ♦ A sequence of finite instructions to be followed for solving a problem
 - Takes a set of values, as input; and
 - Produces a value, or set of values (e.g. sorting), as output.
 - ♦ An algorithm may be given in different forms.
 - A description using English/other languages; or
 - A real computer program, e.g. C++; or
 - A pseudo-code, C-like program, program-language-like program.
- Data structures
 - ♦ The ways of organizing data
- Program = algorithms + data structures

Algorithm Analysis

- Why need algorithm analysis?
 - ♦ Correctness: Just writing a syntax-error-free program is not enough. We need to know whether the algorithm is correct or not, i.e. whether it can give correct answers for the inputs (or called instances).
 - ♦ Efficiency: If the program is run on a large data set, the running time becomes an issue.
 - We want to know how the algorithm performs when the input size is large. The program may be computationally inefficient, or may need lots of memory. We analyze the resources that the algorithm requires: memory, and computation time.
 - We can also compare algorithms without implementations.
 - ♦ Flexibility: We analyze algorithms, rather than problems. A problem can be solved with several algorithms, some are more efficient than others.

Algorithm Analysis (cont'd)

- It is important to know: how much of a particular resource, such as time or storage, is required for a given algorithm
- Methods
 - ♦ Experiment and estimation
- Space complexity
 - ♦ How much space is required
- Time complexity
 - ♦ How much time does it take to run the algorithm
- Often, we deal with estimates!

Space Complexity

- Space complexity = The amount of memory required by an algorithm to run to completion
 - ♦ Instructions
 - ♦ Data
 - ♦ Environment stack
- $S(n)$: the function of the problem size n
- Asymptotic space complexity

Time Complexity

- Often more important than space complexity
 - ♦ Space available (for computer programs!) tends to be larger and larger
 - ♦ Time is still a problem for all of us
 - Algorithms running time **is** an important issue
 - Characterize an algorithm as a function of the “problem size”
 - ♦ Several factors affect the running time:
 - Compiler, computer, algorithm and input data
- E.g.
- Input data = array $\rightarrow$ problem size is N (length of array)
 - Input data = matrix $\rightarrow$ problem size is $N \times M$

Time Complexity (cont'd)

- For this course, it is generally assumed that instructions are executed one after another (sequential), with no concurrent operations $\Rightarrow$ no parallel computers.
- Each operation (e.g. $+$, $-$, $\times$, $/$, $=$) and each memory access (内存訪問) take one run-time unit. Loops and functions can take multiple time units.

Time Complexity (cont'd)

- The size of input is the main consideration
 - ♦ Two functions:
 - $T_{\text{avg}}(N)$: the average-case running time used by an algorithm on input size N ;
 - $T_{\text{worst}}(N)$: the worst-case running time used by an algorithm on input size N .
 - $T_{\text{avg}}(N) \leq T_{\text{worst}}(N)$
 - ♦ $T_{\text{worst}}(N)$ is required in general – it provides a bound for all inputs, including the particularly bad input, which an average-case analysis doesn't provide.

Algorithm's efficiency

- We generally discuss the algorithm's efficiency as a function of the number of elements "N" to be processed (or called problem size)
- The general format is

$$f(N) = \text{efficiency}$$

To calculate the number of the key operations.

- Exact running time $f(N)$ is difficult to find for some cases. It is easier to find the upper and lower bounds of $f(N)$.
- Therefore, we are concerned with its general order of magnitude about the growth of $f(N)$.

A Simple Example

- Calculate the key operations of the algorithm

Sum function
to calculate
the polynomial

$$\sum_{i=1}^N i^3$$

```
int sum(int n)
{
    int partialSum;

    1  partialSum=0;
    2  for (int i=1;i<=n;i++)
    3      partialSum += i*i*i;
    4  return partialSum;
}
```

1

2N+2 (=1 + (N+1) + N)

4N

1

- Lines 1 and 4 count for one unit each.
- Line 3: executed N times, each time four units.
- Line 2: (1 for initialization, N+1 for all tests, N for all increments) total 2N + 2.
- Total time units: 6N + 4. ($\Rightarrow O(N)$, will be discussed later.)

Algorithm's efficiency

- Example

algorithm a : $c_1n^2+c_2n$

algorithm b : c_3n

when n is big enough, algorithm b will be faster than a

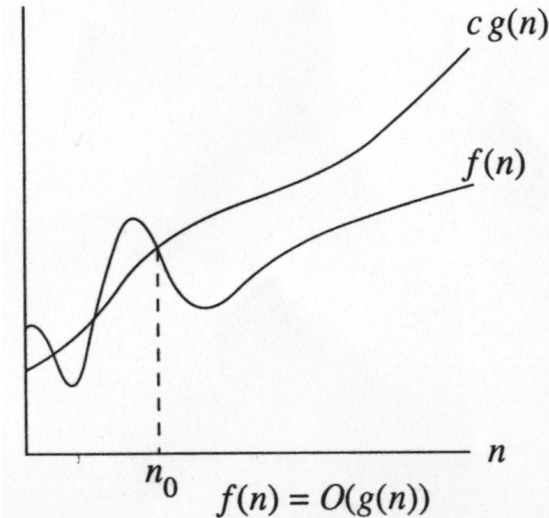
- Asymptotic notation

In 1892, P. Bachmann invented a notation for characterizing the asymptotic behavior of functions. His invention has come to be known as *Big-Oh notation*.

Big-Oh Notation

$$f(N) = O(g(N))$$

- Given functions $f(N)$ and $g(N)$, we say that $f(N)$ is $O(g(N))$ if and only if there are positive constants c and n_0 such that
$$f(N) \leq c g(N) \text{ for } N \geq n_0$$
- The growth rate of $f(N)$ is less than or equal to the growth rate of $g(N)$. Therefore, $g(N)$ is an upper bound on $f(N)$.
- In other words, $f(N)$ grows no faster than $g(N)$ for N . $f(N)$ has order at most $g(N)$.



Big-Oh Notation (cont'd)

- Example

Consider the function $f(n)=8n+128$, we wish to show $f(n) = O(n^2)$.

According to Definition, in order to show this we need to find an integer n_0 and a constant $c>0$ such that for all integers $n \geq n_0$, $f(n) \leq cn^2$.

It does not matter what the particular constants are -- as long as they exist!

For example,

suppose we choose $c=1$. Then

$$\begin{aligned} f(n) \leq cn^2 &\Rightarrow 8n + 128 \leq n^2 \\ &\Rightarrow 0 \leq n^2 - 8n - 128 \\ &\Rightarrow 0 \leq (n - 16)(n + 8). \end{aligned}$$

So, we have that for $c=1$
and $n_0 = 16$, $f(n) \leq cn^2$ for all
integers $n \geq n_0$. Hence, $f(n) = O(n^2)$.

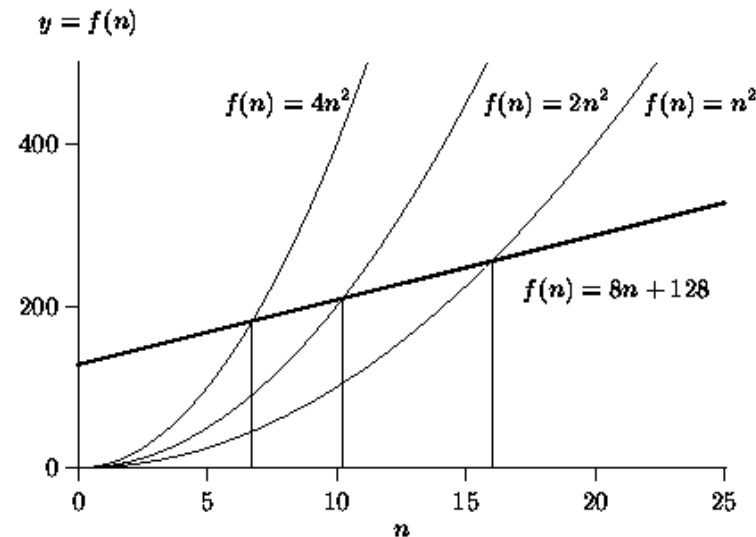


Figure: Showing that $f(n) = 8n + 128 = O(n^2)$.

Big-Oh Notation (cont'd)

- The Big-Oh notation can be derived from $f(n)$ using the following steps:
 - 1) Set the coefficient of each term to 1
 - 2) Keep the largest term in the function and discard the lower order terms.

$$f(n) = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

First, we set all coefficients to 1.

This gives us: $n^2 + n$

Next, we keep the largest term in the function and discard all others.

This leaves: n^2

Therefore, we can write the big-Oh notation as: $f(n) = O(n^2)$

Some rules

- If $T_1(N) = O(f(N))$ and $T_2(N) = O(g(N))$, then
 - ♦ $T_1(N) + T_2(N) = \max(O(f(N)), O(g(N)))$,
 - ♦ $T_1(N) * T_2(N) = O(f(N) * g(N))$
- $\log^k N = O(N)$ for any constant k .

This tells us that logarithms grow very slow.

Typical Growth Rates

Function	Name
c	Constant
$\log N$	Logarithmic
N	Linear
$N \log N$	
N^2	Quadratic
N^3	Cubic

Figure 2.1 Typical growth rates

$\log N$	N	$N \log N$	N^2	N^3	2^N
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65536
5	32	160	1024	32768	4294967296

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n)$$

Running Time Calculations: general rules

- For-loops
 - ♦ Running time is related to the running time of the statements inside the for-loop (including tests) times the number of iterations.

```
int sum(int n)
{
    int partialSum;

    1  partialSum=0;
    2  for (int i=1;i<=n;i++)
    3      partialSum += i*i*i;
    4  return partialSum;
}
```

1

$2N+2 (=1 + (n+1) + n)$

$4n$

1

$4n+2n+2$

Running Time Calculations: general rules (cont'd)

- Nested for-loops

```
for (i=0;i<n;i++) 2n+2 (=1 + (n+1) + n)
    for (j=0;j<n;j++) 2n+2 (=1 + (n+1) + n)
        k++;          n
```

$$2n+2+n (2n+2+n) = 3n^2 + 4n + 2$$

- ♦ Running time = the running time of the statements multiplied by the product of the sizes of all the for-loops.
- ♦ $O(N^2)$

Running Time Calculations: general rules (cont'd)

- Consecutive statements

```
for (i=0;i<n;i++)          2n+2 (=1 + (n+1) + n)
    a[i]=0;                n
for (i=0;i<n;i++)          2n+2 (=1 + (n+1) + n)
    for (j=0;j<n;j++)      2n+2 (=1 + (n+1) + n)
        a[i] += a[j]+i+j;  4n
```

$$(2n+2+n)+(2n+2)+n(2n+2+4n)= 6n^2 + 7n + 4$$

- ♦ These just add two for-loops
(first for-loop + second nested for-loop)
- ♦ $O(N) + O(N^2) = O(N^2)$
Decided by the high-order

Running Time Calculations: general rules (cont'd)

- if/else

```
if ( test )
```

```
    S1
```

```
else
```

```
    S2
```

$O(\text{test} + \max(S1, S2))$

- ♦ Running time = never more than the running time of the test plus the larger of the running times of S1 and S2.

Big-Omega: asymptotic lower bound

Definition:

Consider a function $f(n)$ which is non-negative for all integers $n \geq 0$.

$f(n) = \Omega(g(n))$ (read as “f of n is big-omega of g of n”) if:

There exists positive constants c and n_0 such that

$$f(n) \geq c * g(n) \text{ for all integers } n \geq n_0.$$

And we say $g(n)$ is an asymptotic lower bound of $f(n)$.

Example:

Consider $f(n) = 2n^2 - 5n + 6$. Then $f(n) = \Omega(n^2) = \Omega(n) = \Omega(1) \neq \Omega(n^3)$

Big-Theta: asymptotic tight bound

Definition:

Consider a function $f(n)$ which is non-negative for all integers $n \geq 0$.

$f(n) = \Theta(g(n))$ (read as “ f of n is Big-Theta of g of n ”) if:

There exists positive constants c_1 , c_2 and n_0 such that

$$c_1 * g(n) \leq f(n) \leq c_2 * g(n) \text{ for all integers } n \geq n_0.$$

And we say $g(n)$ is an asymptotic tight bound of $f(n)$.

Generally,

$$f(n) = \Theta(g(n)) \Leftrightarrow \begin{cases} f(n) = O(g(n)) \\ f(n) = \Omega(g(n)) \end{cases}$$

Example:

Consider $f(n) = 2n^2 - 3n + 6$. Then $f(n) = \Theta(n^2)$

To prove that mathematically: We only have to prove $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ respectively, where $g(n) = n^2$.

Rule: If $T(N)$ is a polynomial of degree k , then $T(N) = \Theta(N^k)$

Conclusion

Order of Growth

- To measure the time and space cost (complexity)
- N usually denotes the size of the input.
 - ♦ Size means the total number of bits/bytes/words.
 - ♦ When discussing complexity, we need to clarify the variable it is relative to.
- $O(1)$ $\sim$ constant
 $O(N)$ $\sim$ linear
 $O(N^c)$ $\sim$ polynomial
 $O(\log N)$ $\sim$ logarithmic
 $O(c^N)$ $\sim$ exponential $(c > 1)$

Conclusion (cont'd)

Order of Growth

- $T(N) = O(f(N))$: when N is large enough, $T(N) \leq cf(N)$.
- $T(N) = \Omega(g(N))$: when N is large enough, $T(N) \geq cg(N)$.
- $T(N) = \Theta(h(N))$: when $T(N) = \Omega(h(N))$ and $T(N) = O(h(N))$.

Running Time

- Primitive operations are assumed $O(1)$ time.
- loop / nested-loop / if-else

Written Assignment 01

- Textbook "Data Structures and Algorithm Analysis in C, Mark Allen Weiss"
- Chapter 2
Exercises 2.1, 2.2, 2.4, 2.6
- Please submit via Moodle